

Evals 就是给大模型应用建立一套“自动化考试系统”：你先告诉系统什么叫答得好，再准备一批考题和标准答案，然后让模型批量作答，最后看它通过了多少、错在哪里、该怎么优化。

1. 为什么需要 evals?

文件开头强调，evals 用来测试模型输出是否符合你指定的 **风格标准** 和 **内容标准**。尤其是在你升级模型、尝试新模型、修改 prompt、调整应用逻辑时，evals 是构建可靠 LLM 应用的重要组成部分。

通俗理解：

你不能只靠“我试了几个例子，感觉还不错”来判断一个大模型应用是否可靠。因为 LLM 的输出有随机性，而且真实用户输入会非常复杂。

所以你需要一套固定测试集，每次改 prompt、换模型、改代码，都跑一遍，看结果有没有变好或变坏。

这就像软件工程里的自动化测试：

传统软件测试函数是否返回正确结果；
LLM evals 测试模型回答是否满足预期。

文件也提到，这个过程有点像 **BDD，行为驱动开发**：先定义系统应该怎么表现，再实现和测试。

2. OpenAI evals 的三步流程

文件将构建和运行 evals 概括为三步：

1. **Create an eval for a task**：为一个具体任务创建 eval；
2. **Run your eval with test inputs**：用测试输入运行 eval；
3. **Analyze the results, then iterate**：分析结果，然后改进 prompt。

通俗解释就是：

先定义“考什么”；
再准备“考题和答案”；
最后批改“模型考得怎么样”。

文件用的例子是：

让模型把 IT 支持工单分类为三类：

- Hardware：硬件问题；
- Software：软件问题；
- Other：其他问题。

比如：

“My monitor won’t turn on!”
应该分类为 Hardware。

3. 创建 eval：要定义两个核心东西

文件里说，一个 eval 需要两个关键组成部分：

1) data_source_config：测试数据长什么样

它定义测试数据的 schema，也就是每条测试数据应该包含哪些字段。

在文件示例中，每条数据包含：

- ticket_text：工单文本；
- correct_label：人工标注的正确分类。

通俗理解：

这一步是在规定“试卷格式”。

例如每道题都必须长这样：

```
{  
  "ticket_text": "My monitor won't turn on!",  
  "correct_label": "Hardware"  
}
```

其中 ticket_text 是题目， correct_label 是标准答案。

2) testing_criteria：怎么判断模型答得对不对

在示例中，OpenAI 使用了一个 string_check grader，判断模型输出是否和人工标签完全一致。

也就是说，如果模型输出：

```
Hardware
```

而标准答案也是：

```
Hardware
```

那就通过。

如果模型输出：

```
It seems to be a hardware problem.
```

虽然语义上对，但因为要求“只输出一个词”，它可能就不满足精确匹配规则。

通俗理解：

`testing_criteria` 就是“判卷规则”。

在简单分类任务里，可以用字符串精确匹配；
但在复杂任务里，可能需要更多评判方式，比如：

- 回答是否忠实于上下文；
- 是否覆盖关键点；
- 是否格式正确；
- 是否避免幻觉；
- 是否符合语气要求。

4. 测试数据：最好用 JSONL 文件

文件建议可以上传一个 JSONL 文件作为测试数据。示例中每一行是一条测试样本：

```
{ "item": { "ticket_text": "My monitor won't turn on!", "correct_label":  
"Hardware" } }  
{ "item": { "ticket_text": "I'm in vim and I can't quit!", "correct_label":  
"Software" } }  
{ "item": { "ticket_text": "Best restaurants in Cleveland?", "correct_label":  
"Other" } }
```

这些数据同时包含：

- 模型要处理的输入；
- 人工给出的 ground truth，也就是标准答案。

通俗理解：

这就是给模型准备一套“模拟真实业务场景的题库”。

如果你在做 RAG 应用，这个测试集可能包含：

- 用户问题；
- 应该检索到的文档；
- 标准答案；
- 不允许编造的事实边界。

如果你在做 Agent 应用，测试集可能包含：

- 用户任务；
- 期望调用的工具；
- 允许的步骤；
- 最终结果是否完成。

5. 运行 eval：让模型批量作答

创建 eval 和上传测试数据后，就可以创建一次 eval run。

在 run 里，需要指定：

- 用哪个模型，例如 gpt-4.1 ；
- 用什么 prompt；
- 测试数据来源；
- 如何把测试数据动态填入 prompt。

文件中使用了双大括号模板语法：

```
{{ item.ticket_text }}
```

意思是：每次运行时，把当前测试样本里的 ticket_text 填进 prompt。

通俗理解：

你写一个通用 prompt:

请把下面的工单分类为 Hardware、Software 或 Other。

然后 eval 系统会自动把每条测试数据塞进去，让模型一条条回答。

这比人工复制粘贴测试高效很多，也更可重复。

6. 分析结果：看通过率、失败数、token 用量和报告链接

运行完成后，API 会返回结果信息，包括：

- 总测试条数；
- 通过多少；
- 失败多少；
- 出错多少；
- 每个测试标准的通过情况；
- 模型调用次数和 token 使用情况；
- `report_url`，可以在 dashboard 里可视化查看结果。

文件示例中，一共 3 条测试：

```
"total": 3,  
"errored": 0,  
"failed": 0,  
"passed": 3
```

说明模型在这个小测试集上全部通过。

但文件也提醒，真实业务中通常需要更多 criteria、更多 prompt 版本、不同数据集反复测试。

通俗理解：

不要看到 3/3 通过就觉得系统很可靠。

真实 eval 的价值在于：

- 看哪些类型的问题容易错；
- 看 prompt 改完后有没有提升；
- 看换模型后性能有没有退化；
- 看成本和准确率是否平衡；
- 看是否存在边界场景失败。

7. 这份文件真正想教你的方法论

这份 OpenAI 文档不是只教 API 调用，它背后的核心方法论是：

从“主观感觉”转向“可度量优化”

过去我们常说：

| 这个 prompt 好像效果不错。

但 evals 希望你变成：

| 在 100 条代表性样本中，新 prompt 的通过率从 78% 提升到 91%，其中分类错误下降最多，但边界类样本仍然容易失败。

这就是从“感觉不错”到“工程化评估”。

8. 对你做 AI 应用 / Agent / RAG 项目的启发

结合你之前在学 RAGAS、Agent Evaluation、LangSmith 类评估实践，这份文件可以放在一个更大的框架里理解：

OpenAI Evals 解决的是：如何把评估流程工程化

它关注的是：

- 如何定义 eval；
- 如何准备测试数据；
- 如何设置 grader；
- 如何批量运行；
- 如何查看结果；
- 如何迭代应用。

RAGAS 更关注的是：RAG 任务该用哪些指标

比如：

- context precision；
- context recall；
- faithfulness；
- answer relevancy。

LangSmith / Agent eval 更关注的是：链路追踪和复杂任务归因

比如：

- Agent 有没有调用正确工具；
- 多步推理在哪里失败；
- 检索失败还是生成失败；
- prompt 问题还是工具问题。

所以你可以这样理解：

OpenAI Evals 是“评估执行框架”；
RAGAS 是“RAG 指标体系”；
LangSmith 是“链路观测和调试平台”；
你自己的测试集和业务标准，才是 eval 真正的灵魂。

9. 最通俗的类比

可以把 evals 想象成你给大模型应用设计的一套“高考系统”：

组成部分	在 evals 里是什么	通俗解释
考试科目	task / eval name	要测试什么能力
试卷格式	data_source_config	每道题有哪些字段
标准答案	correct_label / ground truth	人工认可的正确结果
判卷规则	testing_criteria / grader	怎么判断对错
考生	model + prompt	被测试的大模型应用
考试过程	eval run	批量跑测试
成绩单	result_counts / report_url	通过率、失败项、成本

10. 重点记住的 5 个概念

1. **Eval 不是临时测试，而是长期回归测试**
每次改 prompt、换模型、改链路，都应该重新跑。
2. **测试数据比工具更重要**
没有高质量、代表性的测试集，eval 结果没有意义。

3. Grader 决定了你优化的方向

简单任务可以用字符串匹配，复杂任务需要更细的评分标准。

4. Ground truth 是评估的锚点

人工标注的正确答案、正确文档、正确工具调用，是评估可信度的基础。

5. Eval 的目标不是证明模型完美，而是发现失败模式

真正有价值的是知道模型在哪类问题上错、为什么错、怎么改。

小结

这份文件的核心是教你用 OpenAI Evals API 搭建一套自动化评估流程：**定义任务** → **准备数据** → **设置评分标准** → **运行测试** → **分析结果** → **迭代优化**。它特别适合用于 **prompt 调优、模型升级验证、RAG/Agent 应用上线前测试**，以及防止新版本引入质量退化。